

Module 3 — Customization

Lesson 3.5

Statuslines and keybindings

Two low-effort tweaks that punch above their weight. A statusline shows what matters at a glance; a few rebindings cut friction you don't notice until it's gone.

Why it matters

You'll glance at Claude Code thousands of times a week. Customizing what's visible (statusline) and how you input (keybindings) is high-leverage even though it feels cosmetic.

Statusline

The statusline is a thin line of text rendered by Claude Code, typically showing model name, working directory, and other state. You can replace it with the output of a script.

Configuration

In `settings.json`:

```
{
  "statusLine": {
    "type": "command",
    "command": "node ~/.claude/statusline.js"
  }
}
```

The command is run periodically and its stdout becomes the statusline.

A useful statusline script

```
#!/usr/bin/env node
// ~/.claude/statusline.js
const { execSync } = require('child_process');
const path = require('path');

const cwd = path.basename(process.cwd());
let branch = '';
try { branch = execSync('git rev-parse --abbrev-ref HEAD', { stdio: ['pipe', 'pipe', 'ignore']
}).toString().trim(); } catch {}

const model = process.env.CLAUDE_MODEL || '';
const parts = [
  cwd,
  branch && `git:${branch}`,
  model && `[${model}]`,
].filter(Boolean);

process.stdout.write(parts.join(' · '));
```

Make it executable and reference its absolute path.

Setting it up via skill

Claude Code ships a `/statusline-setup` agent in some versions that walks you through generating one. If available:

```
> /statusline-setup
```

Things people put in statuslines

- Current model

- Working dir basename
- Git branch and dirty indicator
- Active context window usage (if your shell can compute it)
- A "do not disturb" indicator when on a release branch
- Cost-so-far in this session (when the SDK / CLI exposes it)

Keep it under ~80 characters so it doesn't wrap.

Keybindings

Claude Code's interactive mode has a default set of keybindings. The most-used:

Default	Action
Enter	Send
Shift+Enter (or Esc+Enter)	Newline in input
Shift+Tab	Cycle permission mode
Esc	Cancel current turn / dismiss UI
Ctrl+C (twice)	Exit session

Customizing

Keybindings live in `~/.claude/keybindings.json` (presence and exact shape varies by version — confirm with the `keybindings-help` skill or `/help`).

A minimal example, rebinding submit to `Ctrl+Enter` (and freeing Enter for newlines):

```
{
  "submit": ["Ctrl+Enter"],
  "newline": ["Enter"]
}
```

Use cases worth considering:

- **Move submit off Enter** if you find yourself accidentally sending mid-thought. Many devs rebind to `Ctrl+Enter` (closer to the Slack/Discord model).
- **Add a chord for `/clear`** — e.g., `Ctrl+K` to wipe context without typing.
- **Add a chord for plan-mode toggle** if `Shift+Tab` is awkward on your layout.

If you have the `keybindings-help` skill installed:

```
> /keybindings-help
```

walks you through changes interactively.

When to bother

Customize after a week of normal use, not on day one. You don't yet know what bugs you. After a week:

- Notice the gestures you do reflexively. Are any awkward?
- Notice what you wish were visible at a glance. Add it to the statusline.
- Notice mis-sends. Move submit off Enter.

Try it

- 1 Write a statusline script that shows just `<dir> · git:<branch>`. Wire it in.
- 2 Use Claude Code for a session. Did you reach for any info that wasn't visible? Add it.
- 3 If accidental sends are a pain, rebind submit to Ctrl+Enter and use it for an afternoon. Stick with it or revert.

Gotchas

- **Statusline scripts run frequently.** Slow scripts make the UI feel laggy. Keep them under ~100ms.
- **Don't shell out to network calls in the statusline.** Cache, or skip.
- **Keybinding changes only apply to new sessions** (in most versions).
- **OS-level shortcuts can shadow** keybindings. Make sure your chord isn't already grabbed by your WM/terminal.

Further reading

- The `statusline-setup` and `keybindings-help` skills (if available in your install)
- Module 7 Automation — for headless mode where statuslines don't apply

Next

→ 3.6 Output styles